

DESIGN & ANALYSIS OF ALGORITHMS (DAA)

Course Code: 23CS302 • Semester: 3-1 • Department of Computer Science & Engineering

Course Scope & Objectives: Algorithmic paradigms, divide-and-conquer, greedy optimizations, dynamic programming, backtracking, branch-and-bound, and NP-completeness.

Unit 1: Algorithm Framework & Divide-and-Conquer

Key Syllabus Topics: Performance Analysis, Recurrence Relations (Master Theorem, Substitution, Recursion Tree), Divide and Conquer: Binary Search, Merge Sort, Quick Sort (Best/Worst case), Strassen's Matrix Multiplication.

Theoretical Framework & Concepts:

Master Theorem provides closed-form asymptotic solutions for divide-and-conquer recurrences $T(n) = aT(n/b) + f(n)$. Strassen reduces matrix multiplication from $O(n^3)$ to $O(n^{2.81})$ using 7 recursive sub-matrix multiplications.

Implementation / Key Formula Reference:

// Master Theorem: If $f(n) = \Theta(n^c)$ where $c = \log_b(a)$, then $T(n) = \Theta(n^c \cdot \log n)$

Frequently Asked University Exam Questions:

• **Q1: Solve $T(n) = 2T(n/2) + n$ using Master Theorem.**

Solution: A: Here $a=2$, $b=2$, $\log_b(a) = \log_2(2) = 1$. Since $f(n) = n = \Theta(n^1)$, Case 2 applies: $T(n) = \Theta(n \log n)$.

Unit 2: Greedy Method Paradigms

Key Syllabus Topics: Greedy Choice Property, Optimal Substructure, Fractional Knapsack, Job Sequencing with Deadlines, Minimum Spanning Trees (Prim's and Kruskal's), Single Source Shortest Path (Dijkstra's), Huffman Coding.

Theoretical Framework & Concepts:

Greedy algorithms make locally optimal decisions at each stage with no backtracking. Huffman coding produces variable-length prefix-free binary codes based on symbol frequency, minimizing compressed bit length.

Implementation / Key Formula Reference:

// Fractional Knapsack: Sort items by value/weight ratio in descending order

Frequently Asked University Exam Questions:

• **Q1: Differentiate between Fractional Knapsack (Greedy) and 0/1 Knapsack (DP).**

Solution: A: Fractional knapsack allows taking fractions of items (Greedy yields optimal $O(N \log N)$ solution); 0/1 knapsack requires discrete decisions, solvable via Dynamic Programming in $O(NW)$ pseudo-polynomial time.

Unit 3: Dynamic Programming

Key Syllabus Topics: Overlapping Subproblems, Optimal Substructure, Memoization vs Tabulation, 0/1 Knapsack, Longest Common Subsequence (LCS), Matrix Chain Multiplication (MCM), Bellman-Ford Shortest Path, Floyd-Warshall All-Pairs Shortest Path.

Theoretical Framework & Concepts:

Dynamic Programming stores solved subproblem solutions in a table, avoiding redundant exponential recomputations. Floyd-Warshall computes all-pairs shortest paths in $O(V^3)$ time via dynamic matrix updates.

Implementation / Key Formula Reference:

```
// LCS Recurrence:
// if (X[i] == Y[j]) dp[i][j] = 1 + dp[i-1][j-1];
// else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
```

Frequently Asked University Exam Questions:

- **Q1: Explain Matrix Chain Multiplication (MCM) and write its recurrence.**

Solution: A: Computes minimum scalar multiplications to multiply a chain of matrices. Recurrence: $m[i,j] = \min_{i \leq k < j} (m[i,k] + m[k+1,j] + p_{i-1} * p_k * p_j)$.

Unit 4: Backtracking & Branch-and-Bound

Key Syllabus Topics: State Space Trees, N-Queens Problem, Sum of Subsets, Graph Coloring (m-colorability), Hamiltonian Cycles, Branch and Bound: 0/1 Knapsack, Traveling Salesperson Problem (TSP) with Reduced Cost Matrix.

Theoretical Framework & Concepts:

Backtracking systematically searches state spaces, pruning invalid subtrees when constraints are violated. Branch and Bound uses bounding functions (e.g. lower bounds for minimization) to prune subtrees in optimization problems.

Implementation / Key Formula Reference:

```
// N-Queens: Safe if no queen shares same column or diagonal |row1-row2| == |col1-col2|
```

Frequently Asked University Exam Questions:

- **Q1: Explain the 4-Queens and 8-Queens backtracking algorithm.**

Solution: A: Places queens row by row; if queen at column c conflicts with previous queens diagonally or vertically, the algorithm backtracks to adjust earlier rows.

Unit 5: NP-Completeness and Approximation Algorithms

Key Syllabus Topics: Class P, Class NP, NP-Hard, NP-Complete, Polynomial-Time Reductions, Cook-Levin Theorem, Proving NP-Completeness (3-SAT to Clique to Vertex Cover), Approximation Algorithms (Vertex Cover, Metric TSP).

Theoretical Framework & Concepts:

Class P contains decision problems solvable in polynomial time; NP contains problems verifiable in polynomial time. NP-Complete problems are the hardest problems in NP; if any NPC problem is in P, then $P = NP$.

Implementation / Key Formula Reference:

// Cook-Levin Theorem: SAT (Boolean Satisfiability) is NP-Complete

Frequently Asked University Exam Questions:

• **Q1: Define P, NP, NP-Hard, and NP-Complete with a Euler diagram description.**

Solution: A: P: Solvable in $O(n^k)$. NP: Verifiable in $O(n^k)$. NP-Hard: At least as hard as any problem in NP.
NP-Complete: In NP and NP-Hard simultaneously.